Deep Learning - Linear Regression: Features, Weights, Bias, MSE

Refai Arafat Rossi

October 2026

1 Introduction

[Chapter 5.1.4: Linear Regression]Linear Regression: Features, Weights, Bias & MSE Based on Chapter 5.1.4, *Deep Learning* (Ian Goodfellow et al.) Master's Level Lecture Deck Department of Computer Science / Data Science October 10, 2026

height

1. Problem: Predict Deep Learning Model Training Time Suppose we want to estimate how many minutes a model will take to train based on two observed features:

- x_1: Number of training epochs
- x_2: Batch size measured in units of 16 samples

We decide to model the relationship using a **Linear Regression Model**:

$$\hat{y} = w_1 x_1 + w_2 x_2 + b$$

where $\hat{y}$ is predicted training time (minutes), w_1, w_2 are weights, and b is the bias.

Learned Model Example Suppose our algorithm has learned parameters $w_1 = 2, w_2 = 3, b = 5$:

$$\hat{y} = 2x_1 + 3x_2 + 5$$

2. What Is the Machine Learning Task? A machine learning task specifies what the model is supposed to do. Here, the task is mapping inputs to continuous real values:

$$\mathbf{x} \longrightarrow \hat{y} \in R$$

0.5 Regression Task ($\hat{y} \in R$) Output is a continuous variable:

- Model training time (minutes)

- Housing prices ($)
- GPU operating temperature ($^{\circ}C$)

0.5 Classification Task ($\hat{y} \in \{0, 1, \ldots\}$) Output is a discrete categorical label:

- Image identification (Cat vs. Dog)
- Medical diagnosis (Benign vs. Malignant)

3 & 4. Features and Targets Feature ($\mathbf{x}$) An input variable/attribute passed to the model to aid prediction.

$$x_1 = numberofepochs, \quad x_2 = batch - sizeunits$$

Target (y) The true ground-truth value we aim to predict:

$$y = actualobservedtrainingtime(minutes)$$

Example Instance: If an experiment uses $x_1 = 10$ epochs and $x_2 = 4$ batch units with ground truth $y = 37$ minutes:

$$\hat{y} = 2(10)+3(4)+5 = 20+12+5 = \mathbf{37} Error(y-\hat{y}) = 37-37 = 0 \quad (Perfectprediction!)$$

5. What Is a Feature Vector? Instead of handling variables individually, we package inputs into a **feature vector x**:

$$\mathbf{x} = x_1 x_2 = 104$$

General n-dimensional Feature Vector For an observation with n input features:

$$\mathbf{x} = x_1 x_2 \vdots x_n \in R^n$$

Each sample in a dataset is represented mathematically as a point in n-dimensional vector space.

6 & 7. Weights and Feature Interpretation Model equation: $\hat{y} = 2x_1 + 3x_2 + 5w_1 = 2$, $w_2 = 3$.

Linear Model Weight Meaning A weight w_j represents the expected change in output $\hat{y}$ per unit change in x_j, holding all other features fixed:

- $w_1 = 2 + 1$ epoch increases predicted time by 2 minutes.
- $w_2 = 3 + 1$ batch-size unit increases predicted time by 3 minutes.

Important Nuance: Weight Magnitude vs. Feature Scale A larger weight does NOT automatically imply a more important feature! Scale matters (e.g., $x_1 = ageinyears$ vs. $x_2 = incomeindollars$). Feature normalization is required before comparing weight magnitudes directly.

8 & 9. Parameters vs. Weights 0.5 Model Parameters (θ) The set of all learnable values adjusted by training algorithms:

$$\theta = \{w_1, w_2, \ldots, w_n, b\}$$

0.5 Structural Decomposition

- **Weights (w_j):** Parameters multiplying features or activations.
- **Bias (b):** Additive parameter shifting output independent of inputs.

$$Weights and biases are specific subsets of model parameters \theta.$$

10. Vector Notation for Linear Regression We write linear regression using vector dot products:

$$\mathbf{w} = w_1 w_2, \quad \mathbf{x} = x_1 x_2 \mathbf{w}^T \mathbf{x} = w_1 w_2 x_1 x_2 = w_1 x_1 + w_2 x_2$$

Vector Formulation of Linear Regression

$$\hat{y} = \mathbf{w}^T \mathbf{x} + b$$

This linear combination is the exact foundational building block used in deep neural network layers.

11 & 12. Linear Regression as an Artificial Neuron 0.5 **Structural Comparison:**

- **Linear Regression:**
$$\hat{y} = \mathbf{w}^T \mathbf{x} + b$$
- **Artificial Neuron:**
$$z = \mathbf{w}^T \mathbf{x} + b, \quad a = f(z)$$

Linear regression is simply a single neuron with an identity activation function ($f(z) = z$).

0.5 [x=1.5cm, y=1.2cm, ¿=stealth] [circle, draw, fill=blue!10] (x1) at (0, 0.8) x_1; [circle, draw, fill=blue!10] (x2) at (0, -0.8) x_2; [circle, draw, fill=green!10, inner sep=3pt] (sum) at (1.5, 0) $\sum$; [draw, fill=yellow!10, minimum height=0.6cm] (bias) at (1.5, -1.2) $+b$; [circle, draw, fill=red!10] (y) at (2.8, 0) $\hat{y}$; [-¿] (x1) – node[above, xshift=-2pt] $\times w_1$ (sum); [-¿] (x2) – node[below, xshift=-2pt] $\times w_2$ (sum); [-¿] (bias) – (sum); [-¿] (sum) – (y);

13–17. Dataset Evaluation Example Consider model $\hat{y} = 2x_1 + 3x_2 + 5$ evaluated on 4 experimental runs:

Exp D exhibits a prediction discrepancy ($\hat{y} = 36$ vs. $y = 35$). We need a quantitative metric to measure overall dataset error.

18–21. Performance Metric: Mean Squared Error (MSE) To quantify model error across m dataset samples, we compute the Mean Squared Error:

$$MSE = \frac{1}{m} \sum_{i=1}^{m} \left(\hat{y}^{(i)} - y^{(i)} \right)^2$$

0.5 **Why square the residuals?**

Exp	Epochs (x_1)	Batch (x_2)	Actual (y)	Predicted ($\hat{y}$)	Error ($y - \hat{y}$)	Squared Erro
A	2	2	15	$2(2) + 3(2) + 5 = 15$	0	0
B	4	3	22	$2(4) + 3(3) + 5 = 22$	0	0
C	6	4	29	$2(6) + 3(4) + 5 = 29$	0	0
D	8	5	35	$2(8) + 3(5) + 5 = \mathbf{36}$	$\mathbf{-1}$	**1**

- Prevents positive and negative errors from canceling out ($+2 + (-2) = 0$).
- Penalizes larger deviations much more severely ($1^2 = 1$ vs. $5^2 = 25$).

0.5 MSE Calculation for Dataset

$$MSE = \frac{0 + 0 + 0 + 1}{4} = \mathbf{0.25}$$

An MSE of 0 represents perfect prediction across all samples.

25 & 26. Residuals and Geometric Interpretation Residual Definition The residual $r^{(i)}$ for sample i is the vertical distance between the actual target value $y^{(i)}$ and the predicted value $\hat{y}^{(i)}$:

$$r^{(i)} = y^{(i)} - \hat{y}^{(i)}$$

0.5 [scale=0.65] [-¿] (0,0) – (5,0) node[right] x; [-¿] (0,0) – (0,5) node[above] y; [thick, blue] (0,1) – (4.5,4.5) node[right] $\hat{y} = wx + b$; [red] (3, 2.3) circle (3pt) node[right] $(x^{(i)}, y^{(i)})$; [dashed, red, thick] (3, 2.3) – (3, 3.33); [left, red] at (3, 2.8) $r^{(i)}$;

0.5 **Key Concept:** Linear regression finds parameters w, b that minimize the average sum of these squared vertical distances across all data points.

27–30. Geometric Meaning and Necessity of Bias Why do we need an explicit bias term b?

0.5 Without Bias ($b = 0$): $\hat{y} = wx$ Forces the line/plane through the origin $(0, 0)$. If target $y = 5$ when $x = 0$, a bias-free model fails completely.

0.5 With Bias ($b \neq 0$): $\hat{y} = wx + b$ Allows affine transformations. Bias shifts the decision boundary / regression hyperplane up or down.

- **1D Line ($\hat{y} = wx + b$):** w dictates slope; b dictates y-intercept.
- **2D Plane ($\hat{y} = w_1x_1 + w_2x_2 + b$):** w_1, w_2 dictate orientation; b shifts plane elevation.

32 & 33. Framing Parameter Learning via Optimization We don't know ideal weights w and bias b beforehand. We start with random parameter initializations and optimize via **Gradient Descent**.

Loss Function to Minimize:

$$J(w_1, w_2, b) = \frac{1}{m} \sum_{i=1}^{m} \left((w_1 x_1^{(i)} + w_2 x_2^{(i)} + b) - y^{(i)} \right)^2$$

Term	Meaning in Linear Regression Context
Task	Predict training runtime given model hyperparameters.
Feature	Individual input attribute (x_1 = epochs, x_2 = batch units).
Feature Vector (x)	Vector packaging all input variables for a single sample.
Target (y)	Ground-truth true real-world observation.
Prediction ($\hat{y}$)	Output generated by model $\mathbf{w}^T\mathbf{x} + b$.
Weight (w)	Multiplicative parameters defining feature contributions.
Bias (b)	Additive parameter enabling affine shift of predictions.
Parameters (θ)	Set of all learnable variables ($\theta = \{\mathbf{w}, b\}$).
Residual (r)	Raw prediction error ($y - \hat{y}$).
MSE	Mean Squared Error measuring performance over dataset.

Gradient Updates:

$$w_j \leftarrow w_j - \eta \frac{\partial J}{\partial w_j}, \qquad b \leftarrow b - \eta \frac{\partial J}{\partial b}$$

$$\text{Input x} \longrightarrow Prediction\hat{y} \longrightarrow MSELossJ \longrightarrow Gradients\nabla J \longrightarrow ParameterUpdates$$

35. Terminology Summary Table

36 & 37. Student Classroom Exercises Exercise 1: Zero Error Calculation Given model $\hat{y} = 3x_1 + 2x_2 + 4$. For sample $\mathbf{x} = [5, 3]^T$ with actual target $y = 25$. Compute prediction, error, and squared error.

Solution:

$$\hat{y} = 3(5) + 2(3) + 4 = 15 + 6 + 4 = \mathbf{25} e = 25 - 25 = \mathbf{0} e^2 = \mathbf{0}$$

Exercise 2: Non-Zero Error Calculation For the same model, given sample $\mathbf{x} = [6, 4]^T$ with actual target $y = 32$.

Solution:

$$\hat{y} = 3(6) + 2(4) + 4 = 18 + 8 + 4 = \mathbf{30} e = 32 - 30 = \mathbf{2} e^2 = \mathbf{4}$$

38. Core Equations to Remember

1. **Linear Regression Prediction Equation:**

$$\underbrace{\hat{y}}_{Prediction} = \underbrace{\mathbf{w}^T}_{Weights} \underbrace{\mathbf{x}}_{Features} + \underbrace{b}_{Bias}$$

2. **Mean Squared Error Objective Function:**

$$MSE = \frac{1}{m} \sum_{i=1}^{m} \left(\hat{y}^{(i)} - y^{(i)} \right)^2$$

3. **Optimization Goal:**

$$\mathbf{w}^*, b^* = \arg\min_{\mathbf{w}, b} MSE(\mathbf{w}, b)$$

39. Final Summary: Linear Model → Deep Neural Network

Progression of Model Complexity:

1. **Linear Regression (Single Identity Neuron):**

$$\hat{y} = \mathbf{w}^T\mathbf{x} + b$$

2. **Single Non-Linear Neuron:**

$$a = f(\mathbf{w}^T\mathbf{x} + b)$$

3. **Multi-Layer Deep Neural Network:**

$$\mathbf{h}^{(1)} = f_1(W^{(1)}\mathbf{x}+\mathbf{b}^{(1)}), \quad \mathbf{h}^{(2)} = f_2(W^{(2)}\mathbf{h}^{(1)}+\mathbf{b}^{(2)}), \quad \ldots, \quad \hat{y} = f_L(W^{(L)}\mathbf{h}^{(L-1)}+\mathbf{b}^{(L)})$$

Final Conceptual Takeaway Deep networks are constructed by stacking the fundamental linear regression operation ($\mathbf{w}^T\mathbf{x} + b$) across multiple layers interleaved with non-linear activation functions.